



**Clock Counter Profiling of GPU Memory
Load Cache Modifier Effects:
An Empirical L1→L2→DRAM Hierarchy
Study
on NVIDIA Ampere and Turing GPUs**

Author: [YOUR NAME]

Candidate No.: [CANDIDATE NUMBER]

Degree: BSc Computer Science with Artificial Intelligence

Department: School of Engineering and Informatics

Supervisor: [SUPERVISOR NAME]

Submission year: 2025/26

Statement of Originality

This report is submitted as part of the requirements for the degree of BSc Computer Science with Artificial Intelligence at the University of Sussex. It is the original work of the author except where stated otherwise. All sources used have been acknowledged in the references.

Signed: _____

Date: _____

Acknowledgements

The author would like to thank the project supervisor for guidance and feedback throughout this work. Experiments were conducted using the NVIDIA CUDA toolkit and Nsight Compute profiler. The open-source CuASM assembler [1] was used for PTX-to-SASS instruction decoding.

Summary

Intuition: why sub-instruction profiling matters for AI workloads. As deep-learning training and inference become progressively more computationally demanding, the limiting factor on real GPU hardware has shifted from arithmetic throughput to memory traffic. Recent compiler-and-profiler work makes the per-instruction-cost dimension explicit: CuAsmRL formulates SASS scheduling as RL on per-instruction cost [3] and KPerfIR proposes a per-instruction performance IR [4] — both assume an instrument that resolves per-instruction memory cost but neither builds one. This report builds and validates that instrument, and turns the resulting cost map into actionable guidance: **+38.4%** throughput on a 16 MB DRAM-streaming kernel under `.cs` and **−52.2%** on a 16 KB sub-L1 temporal-reuse kernel under `.cg` (Table 4.7, Section 4.5.3) — magnitudes large enough that modifier choice dominates measured throughput.

The PTX ISA exposes four cache policy modifiers on `ld.global` (`.ca`, `.cg`, `.cs`, `.nc`), described as “*performance hints only*” [5, §8.7.8.1] with verbatim per-modifier definitions given in Section 2.2. No public study isolates all four across the full sub-L1→L2→DRAM working-set range with hardware-counter cross-validation [6, 7, 8, 9].

This report builds the missing instrument. It develops a PTX-level profiling methodology that resolves cache-tier transitions on two NVIDIA GPUs (RTX 3060 Ti, SM86 Ampere; RTX 2080 Ti, SM75 Turing) using `%clock64` (primary 64-bit cycle counter) sampled inside the kernel, with `%clock` read alongside as an ISA-mandated low-32-bit cross-check. On-chip sampling is the load-bearing choice: external trace tools such as `nsys` cannot bracket a single SASS instruction, while the on-chip primitives read the device’s counters directly with the load under test interposed between two reads, addressing the variable-latency-completion pitfall noted in the cycle-accurate CUDA benchmarking literature [10] via a first-consumer pattern (Section 3.2). Cycle-counter readings are cross-validated against NVIDIA Nsight Compute (NCU) hardware sector counters, which classify hits architecturally from L1TEX tag arrays — “*Sector accesses are classified as hits if the tag is present and the sector-data is present within the cache line. Tag-misses and tag-hit-data-misses are all classified as misses*” [11] — so agreement between the two instruments carries weight (Sections 3.7 and 4.1).

The hierarchy is mapped on both architectures. Pointer-chase latency confirms tier boundaries on SM86 at 39 cy (L1), 329–333 cy (L2), 533–547 cy (DRAM); SM75 yields 40 / 232–238 / 358–432 cy. Streaming throughput plateaus at 17.4 / 10.1 / 4.9 GB/s (SM86) and

14.0 / 7.5 / 4.7 GB/s (SM75).

With the instrumentation validated, the four modifiers produce a working-set-conditional performance map (B-reload microbenchmark, design and full parameters in Section 3.4). At sub-L1, .nc and .cs are 26% faster than .ca, and .cg is $2.35\times$ slower (policy bypass to L2). At L2-bound, .cs uniquely retains a 32% advantage; the other three converge. Post-L2-saturation, all four are indistinguishable to within 0.5%. Full numbers: Table 4.4, Figure 4.6.

The map is structurally inaccessible to Triton (Section 4.5). Matmul/attention emit LDGSTS (the SASS form of PTX `cp.async`, which routes to shared memory bypassing the L1 data cache (L1D) and the register file) so there is no LDG for the modifier to attach to (structural dead zone). Elementwise emits LDG.E.CONSTANT which decodes to .nc — already the optimal sub-L1 modifier — so any user override worsens performance (choice-locked dead zone). The map is fully actionable in hand-written CUDA on the same access patterns: .cs on a 16 MB DRAM-bound streaming kernel yields **+38.4%**; .cg on a 16 KB sub-L1 reuse kernel costs **-52.2%** (Table 4.7, Section 4.5.3). The dead zone is a property of the Triton compilation path, not of the GPU hardware.

Contents

Statement of Originality	1
Acknowledgements	2
Summary	3
Nomenclature and Abbreviations	10
1 Introduction	11
1.1 Motivation	11
1.2 Project Objectives	12
2 Background and Literature Review	14
2.1 NVIDIA GPU Memory Hierarchy	14
2.2 PTX Cache Modifier Semantics	15
2.3 Prior Work on GPU Cache Characterisation	16
3 Methodology	17
3.1 Why Two Regimes? Latency vs. Throughput	17
3.2 Clock Instrumentation	18
3.3 The Latency-Regime Design: Pointer-Chase	19
3.4 The Throughput-Regime Design: B-Reload Sweep	20
3.5 Cache Policy Semantics and Predicted Clock-Data Signatures	22
3.6 SASS Validation	23
3.7 NCU as the Independent Confirmation Instrument	24
4 Results	26
4.1 clock64 Validation Against NCU	26
4.2 Hierarchy Latency: Per-LDG Service Time	29
4.3 Hierarchy Throughput: Amortised Per-Element Cost	30
4.4 Modifier Effects in the Throughput Regime	32
4.4.1 Full Four-Modifier Taxonomy	32
4.4.2 Secondary Effect: L2 Bandwidth Saturation	35
4.4.3 Cross-Architecture Comparison	35
4.5 Triton vs. Hand-Written CUDA: Dead Zone vs. Actionable Window	37

4.5.1	Triton Matmul and Attention: Structural Dead Zone	37
4.5.2	Triton Elementwise Kernels: Choice-Locked Dead Zone	37
4.5.3	Hand-Written CUDA Kernels: Modifier Selection Is Actionable . . .	38
4.5.4	Modifier Selection Decision Guide	39
5	Discussion	41
5.1	Is the Methodology Valid?	41
5.2	What Do the Modifier Effects Say About Hardware?	41
5.3	Why Doesn't Triton Expose Them?	41
5.4	Limitations	42
6	Professional Considerations	43
6.1	Ethical Considerations	43
6.2	Legal Considerations	43
6.3	Reproducibility	43
7	Conclusions and Recommendations	44
7.1	Conclusions	44
7.2	Recommendations for Future Work	44
A	Experimental Parameter Reference	48
B	SASS Encoding Reference	49
C	Raw Data Summary Tables	50

List of Figures

2.1	Memory hierarchy architecture for both experimental platforms. Measured bandwidth and latency values from this study’s bandwidth and latency sweeps are annotated alongside SM clock frequencies.	14
4.1	Dual-axis cross-instrument view for both GPUs. Left axis: <code>clock64</code> cycles per iteration for <code>.ca</code> (filled circles) and <code>.cg</code> (squares). Right axis: NCU L1 hit rate for <code>.ca</code> (triangles, dashed). Read it like this: on SM86 (left), the <code>.ca</code> cycle trace spikes 34% at $w = 6$ while the NCU hit-rate trace stays flat at 98.5% — the bank-stall signature visible simultaneously in both instruments. SM75 (right): the L1 saturation collapse at $w = 8$ (WS=64 KB=L1 capacity) is marked.	28
4.2	Pointer-chase latency in cycles per hop vs. chain size. The reclassified mixed L1/L2 boundary point at 128 KB (SM86) is shown in pink. True L2 latency (333 cy) is measured at 512 KB, where the chain cleanly resides in L2. SM75 (right) shows a less pronounced boundary effect because 128 KB is 2× the SM75 L1 capacity.	30
4.3	Hierarchy throughput: cycles per element vs. working-set size for <code>.ca</code> and <code>.cg</code> global loads in the independent-load (pipelined) regime. Background shading indicates the detected tier. Left: SM86 (L1=128 KB, L2=3 MB). Right: SM75 (L1=64 KB, L2=5.5 MB). Read it like this: three distinct plateaus per panel mark the L1, L2, and DRAM service regimes.	31
4.4	Achieved throughput in GB/s vs. working-set size for both GPUs. Read it like this: peak L1 throughput under <code>.ca</code> is substantially higher than under <code>.cg</code> ; the two converge at the L2 and DRAM plateaus where the policy distinction stops mattering.	31
4.5	Two-modifier throughput collapse: <code>.ca</code> vs. <code>.cg</code> cycle counts as a function of active warp count. Bootstrap 95% CI shown (shaded). Read it like this: sub-L1, the two diverge by 2.35×; at the L1 capacity boundary they collapse to indistinguishable. SM86 (left): the $w = 6$ banking-stall anomaly is annotated. SM75 (right): same collapse pattern at $w = 16$ (WS=128 KB > 64 KB L1).	33

4.6	All four cache modifiers across warp counts for SM86 (left) and SM75 (right). Read it like this: <code>.cs</code> (Evict-First) is uniquely visible at L2-bound working sets, where it retains a $\approx 32\%$ cycle-count advantage over the other three.	33
4.7	Modifier performance ranking at sub-L1 (top row, $w = 1$) and L2-bound (bottom row, $w = 16$) working sets for both SM86 (left column) and SM75 (right column). Values labelled as absolute cycles and ratio relative to <code>.ca</code> . Red ratio > 1 means slower than <code>.ca</code> ; green < 1 means faster.	34
4.8	Cross-architecture comparison of latency, bandwidth, and sub-L1 modifier performance between SM86 (Ampere) and SM75 (Turing). Read it like this: the qualitative ranking is identical across architectures; SM86 shows higher L1 bandwidth but higher L2 latency.	36
4.9	Cache modifier selection decision tree for CUDA and Triton kernels, synthesising all experimental results. The tree routes through: (1) instruction type check (<code>LDGSTS</code> vs. <code>LDG</code>), (2) working-set size relative to L1/L2 capacities, and (3) whether the kernel is a Triton elementwise kernel. All Triton paths reach a dead-zone outcome; hand-written CUDA paths reach the actionable region.	39

List of Tables

2.1	Hardware specifications of the two GPU platforms used.	15
2.2	PTX cache modifier to SASS opcode mapping on both platforms, confirmed by <code>cuobjdump</code> disassembly. The <code>.cs</code> mapping to <code>LDG.E.EF</code> is non-obvious and is established by binary inspection.	15
4.1	Cross-instrument validation: NCU hardware counters vs. <code>clock64</code> for SM86 (RTX 3060 Ti). NCU <code>.cg</code> L1 hit is 0.00–0.01% at all conditions, confirming structural L1 bypass architecturally. † At $w = 6$, NCU L1 hit rate is unchanged from $w = 4$ yet <code>clock64</code> records a 34% <code>.ca</code> cycle increase: bank-stall latency detected by timing but invisible to hit-rate counters. . .	29
4.2	Pointer-chase latency per memory tier (corrected, mixed boundary excluded). . .	29
4.3	Peak throughput per memory tier for both GPU platforms (validated clocks). . .	31
4.4	Four-modifier performance summary for SM86 (RTX 3060 Ti) and SM75 (RTX 2080 Ti).	33
4.5	Cross-architecture summary of key measured quantities.	36
4.6	Per-SM working set for Triton elementwise kernels on representative models. All configurations are sub-L1 for both SM86 and SM75.	38
4.7	Cache modifier effect on three realistic CUDA kernel patterns (SM86 RTX 3060 Ti). All values are relative to the <code>.ca</code> baseline. Best modifier per pattern shown in bold	38
A.1	Complete parameter reference for all experiments.	48
B.1	Cache modifier bit encoding in LDG instruction <code>qword2</code> , bits [15:12]. . . .	49
C.1	Four-modifier cache sweep results: SM86 RTX 3060 Ti. Median cycles per iteration with 95% bootstrap CI.	50
C.2	Raw cross-instrument data for SM86: NCU L1 hit rate (both modifiers) and <code>clock64</code> speed-up ratio. See Table 4.1 for annotated summary.	50

Nomenclature and Abbreviations

Term	Definition
CUDA	Compute Unified Device Architecture (NVIDIA parallel computing platform)
PTX	Parallel Thread Execution (NVIDIA virtual ISA)
SASS	Shader Assembly (NVIDIA native GPU machine code)
SM	Streaming Multiprocessor (NVIDIA GPU compute unit)
L1D	Level-1 Data Cache
L2	Level-2 Cache (shared across all SMs on the chip)
DRAM	Device RAM (global off-chip memory; GDDR6 in this study)
WS	Working Set (total bytes actively accessed by a kernel)
LDG	Load Global (SASS instruction class for global memory reads)
LDGSTS	Load Global Store Shared (asynchronous memory copy instruction)
LDG.E.EF	Load Global Evict-First (SASS opcode for <code>.cs</code> modifier)
<code>.ca</code>	Cache-All: fills both L1D and L2 on miss
<code>.cg</code>	Cache-Global: fills L2 only, bypasses L1D
<code>.nc</code>	Non-Coherent (read-only): routes through read-only texture cache
<code>.cs</code>	Cache-Streaming: L1D fill with Evict-First hint
NCU	NVIDIA Nsight Compute (GPU kernel profiler)
CI	Confidence Interval (95% bootstrap, 10 000 resamples)
cy/elem	Cycles per element (throughput metric)
cy/hop	Cycles per pointer hop (latency metric)
GB/s	Gigabytes per second (bandwidth metric)
pp	Percentage points
SM86	NVIDIA Ampere microarchitecture (compute capability 8.6)
SM75	NVIDIA Turing microarchitecture (compute capability 7.5)

Chapter 1: Introduction

1.1 Motivation

Why precise sub-instruction profiling matters for AI kernels

Several recent lines of GPU performance research converge on the same diagnosis: kernel-level analysis is no longer fine enough to explain why memory-bound deep-learning kernels run at the throughput they do. What is missing is not raw measurement but *a per-instruction understanding of why loads slow programs* concrete enough to drive a fix. CuAsmRL formulates SASS scheduling as RL on per-instruction cost [3]; KPerfIR proposes a per-instruction performance IR motivated by kernel-granularity profilers being too coarse for automated optimisation [4]; cycle-accurate CUDA benchmarking demonstrates that on-chip clock counters can resolve per-instruction service time given correct handling of variable-latency loads and scheduler reordering [10]. Each treats per-instruction memory cost as a required input and does not produce the instrumentation: there is no published cost map for the four PTX `ld.global` cache modifiers across the full sub-L1→L2→DRAM range with hardware-counter cross-validation [5]. This report supplies that instrument and turns the cost map into actionable modifier-selection guidance.

Standard profilers do not close this gap. NCU and Nsight Systems aggregate at the kernel/warp level: by the time those metrics are produced, the per-instruction effect has been averaged across thousands of loads. To recover per-instruction cost, instrumentation must be brought inside the kernel, between the load and its first consumer, with variable-latency completion handled explicitly [10].

This report builds that capability. On-chip cycle counters (`%clock64` primary, `%clock` cross-check) are bracketed around individual SASS instructions using a first-consumer pattern, then cross-validated against NCU hardware sector counters (Sections 3.2, 3.7 and 4.1). The two instruments agree at every condition tested except a single $w = 6$ L1 bank-stall regime, where their disagreement reveals a real microarchitectural effect.

Once the per-instruction map is known, it is directly actionable on the access patterns that dominate real AI workloads: **+38.4%** on a 16 MB DRAM-streaming kernel under `.cs` (representative of activation-stream reads and embedding-table lookups) and **−52.2%** on a 16 KB sub-L1 temporal-reuse kernel under `.cg` (representative of small persistent weight tiles), full data in Table 4.7 and Section 4.5.3. The actionable window is hidden

from Triton by structural and choice-locked dead zones (Table 4.6, Section 4.5). The broader direction: (i) establish sub-instruction profiling as a methodology that resolves what kernel-level profilers cannot, and (ii) feed the resulting cost map back into compiler heuristics.

Hardware context

Hierarchy and PTX modifier semantics: Sections 2.1 and 2.2. Two facts the report relies on: (i) each modifier compiles to a distinct SASS opcode (Table 2.2); (ii) modifiers are reachable only via inline PTX or the `__ldca/__ldcg/__ldg` intrinsics, not the standard CUDA C++ API [13]. Triton abstracts modifier selection to the compiler [14, 4]; whether its choices are optimal, and whether expert users can improve on them, are open questions with direct economic consequence.

This report answers both. It rests on five claims:

1. PTX exposes four cache modifiers whose effects on real hardware are not characterised in the public literature.
2. Characterising them requires measuring GPU memory load timing, and there are two physically distinct timing regimes — the service time of one isolated load, and the throughput of many loads in flight together. Each regime needs its own experimental design.
3. Both regimes can be instrumented with `clock64`, but `clock64` alone cannot prove which cache tier served any individual load. NCU hardware counters provide the independent confirmation.
4. With instrumentation validated, the four modifiers produce a working-set-conditional performance map: different rankings at sub-L1, L2-bound, and DRAM-bound regimes.
5. This map is structurally inaccessible to Triton but fully actionable in hand-written CUDA: on representative AI-workload patterns, correct `.cs` yields **+38.4%** (DRAM streaming) and wrong `.cg` costs **−52.2%** (sub-L1 reuse). Full data: Table 4.7, Section 4.5.3.

1.2 Project Objectives

The objectives of this project are:

1. Develop and validate a profiling methodology built on two on-chip timing primitives (`%clock64` primary, `%clock` 32-bit cross-check), with an independent NCU

- hardware-counter cross-check, that reliably resolves cache-tier transitions (Claim 3).
2. Characterise the L1→L2→DRAM latency and bandwidth hierarchy on SM86 Ampere and SM75 Turing using controlled microbenchmarks (Claim 2).
 3. Quantify the working-set-conditional performance of all four PTX modifiers, including secondary effects (L1 bank contention, L2 saturation) (Claim 4).
 4. Confirm the PTX-to-SASS lowering of each modifier by binary disassembly (`cuobjdump` + `CuAssembler` [1]), so timing claims tie to known SASS opcodes (Claims 3 and 4).
 5. Audit and validate the measurement pipeline before reporting modifier results (Claim 3).
 6. Determine whether modifier benefit is reachable from the Triton compilation framework (Claim 5).
 7. Demonstrate via hand-written CUDA kernels (streaming, temporal-reuse, random-gather) that modifier selection is directly actionable outside Triton (Claim 5).
 8. Synthesise the cost map into a decision guide (Figure 4.9) usable by custom-kernel developers (Claim 5).

Chapter 2: Background and Literature Review

2.1 NVIDIA GPU Memory Hierarchy

NVIDIA GPU memory is a three-tier hierarchy above off-chip DRAM (Figure 2.1). Each SM holds a unified L1D + shared-memory partition: 128 KB on Ampere (SM86, default fully L1D) [13, 15]; 96 KB with 64 KB L1D default on Turing (SM75) [13]. All SMs share a single on-die L2 (3 MB on RTX 3060 Ti, 5.5 MB on RTX 2080 Ti). Below L2: GDDR6.

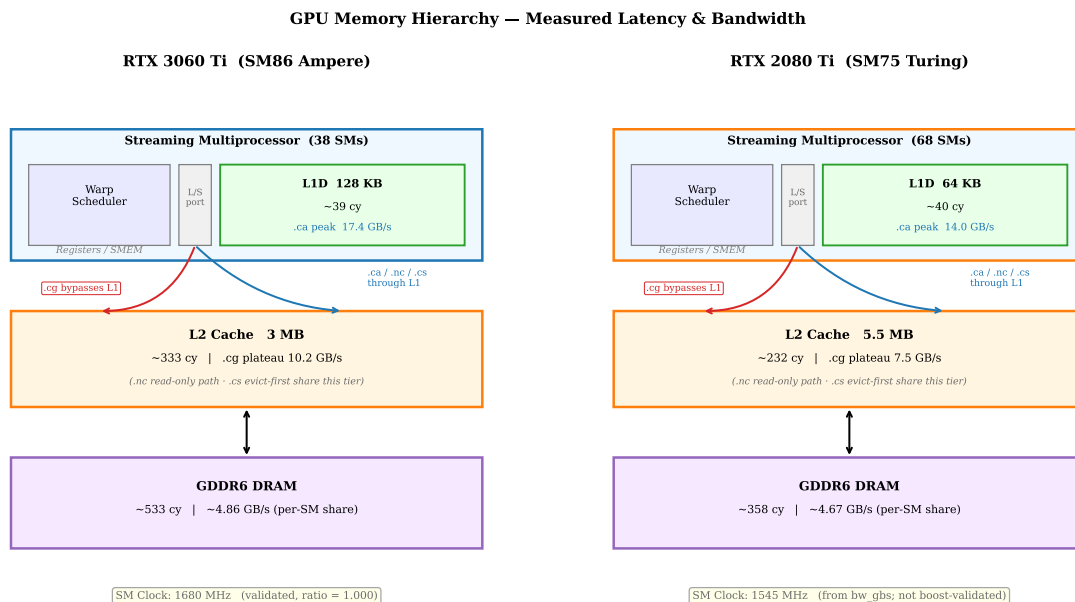


Figure 2.1: Memory hierarchy architecture for both experimental platforms. Measured bandwidth and latency values from this study’s bandwidth and latency sweeps are annotated alongside SM clock frequencies.

Table 2.1 summarises hardware specs, all cross-checked against runtime CUDA device-property queries — this matters because the SM75 L2 size is widely misquoted in prior microbenchmarking work as 4 MB; the actual silicon and runtime value on the RTX 2080 Ti is 5.5 MB, and the wrong value places the L1→L2 boundary at the wrong WS in the sweep (Section 4.3).

Table 2.1: Hardware specifications of the two GPU platforms used.

Property	RTX 3060 Ti (SM86)	RTX 2080 Ti (SM75)
Microarchitecture	Ampere	Turing
Compute Capability	8.6	7.5
SM Count	38	68
L1D Capacity (per SM)	128 KB	64 KB
L2 Capacity	3 MB	5.5 MB
DRAM Type	GDDR6	GDDR6
SM Clock (validated)	1680 MHz	1545 MHz
CUDA Version	12.x	12.x

2.2 PTX Cache Modifier Semantics

The PTX `ld.global` instruction accepts four cache policy qualifiers. The ISA is explicit: they are “*performance hints only*” that “*do not change the memory consistency behavior of the program*” [5]. PTX is an intermediate form; CUDA source is “*first compiled to PTX*”, then “*further optimized and compiled to device code (SASS...) for the specific GPU architecture with Nvidia’s proprietary compiler (*ptxas*)*” [10], which can remap modifiers silently. The only authoritative statement of what the hardware executes is the SASS binary — disassembled for every kernel measured here. Each modifier compiles to a distinct SASS opcode on both architectures (Table 2.2); the `.cs` mapping to `LDG.E.EF` (Evict-First) rather than `LDG.E.STRONG.GPU` is non-obvious and was confirmed by disassembly.

Table 2.2: PTX cache modifier to SASS opcode mapping on both platforms, confirmed by `cuobjdump` disassembly. The `.cs` mapping to `LDG.E.EF` is non-obvious and is established by binary inspection.

PTX modifier	SASS description	SM86 opcode	SM75 opcode
<code>.ca</code>	Cache-All (L1D fill)	<code>LDG.E.STRONG.SM</code>	<code>LDG.E.STRONG.CTA</code>
<code>.cg</code>	Cache-Global (L2-only)	<code>LDG.E.STRONG.GPU</code>	<code>LDG.E.STRONG.GPU</code>
<code>.nc</code>	Non-Coherent (read-only)	<code>LDG.E.CONSTANT</code>	<code>LDG.E.CONSTANT.SYS</code>
<code>.cs</code>	Cache-Streaming (Evict-First)	<code>LDG.E.EF</code>	<code>LDG.E.EF</code>

The PTX ISA gives each modifier a distinct intent. `.ca` caches “*at all levels, likely to be accessed again*” [5, §8.7.8.1]; `.cg` caches “*at global level (cache in L2 and below, not L1)*” [5, §8.7.8.1]; and `.cs` “*allocates global lines with evict-first policy in L1 and L2 to limit cache pollution by temporary streaming data that may be accessed once or twice*” [5, §8.7.8.1]. The `.nc` path (`ld.global.nc`) loads through a “*non-coherent read-only cache*” [5, §8.7.8.1], which NVIDIA characterises with one important caveat: “*on*

some architectures, the texture cache is larger, has higher bandwidth, and longer latency than the global memory cache. For applications with sufficient parallelism to cover the longer latency, `ld.global.nc` should offer better performance” [5, §8.7.8.1]. The phrase “sufficient parallelism” (NVIDIA’s wording, paraphrased here) is left unquantified.

These intents predict, qualitatively, that modifier effects should be working-set-dependent. The detailed predicted clock-data signatures are deferred to Section 3.5, where they sit alongside the experimental design that tests them.

2.3 Prior Work on GPU Cache Characterisation

Several groups have measured the latency and bandwidth profile of NVIDIA GPU caches: Wong et al. [7] introduced the pointer-chase methodology on Fermi (predating PTX ISA 2.0’s four-modifier formalisation); Jia et al. [6] applied it to Volta with a single default modifier; Muaaz et al. [8] extended bandwidth/roofline modelling to Volta, also single modifier; Choquette et al. [16] documented the A100 memory system including the texture path; Yan et al. [9] provided Ampere microbenchmarks using the same `clock64` pointer-chase.

None of these studies satisfies all four criteria of a complete modifier characterisation: (a) all four modifiers swept; (b) the full sub-L1→L2→DRAM range; (c) cycle-counter measurements cross-validated against hardware sector counters; (d) whether modifier selection is reachable from framework code. Triton [14] emits `LDGSTS` for matmul/attention on SM80+; its automatic modifier choices have not been evaluated against an empirical optimum, and the dead-zone structure has not been identified.

Chapter 3: Methodology

3.1 Why Two Regimes? Latency vs. Throughput

Claim Advanced by This Section

Claim 2. Characterising cache modifiers requires measuring GPU memory load timing, and there are two physically distinct timing regimes. A single isolated load has a service time. Many loads issued back-to-back overlap in the pipeline and amortise into a throughput. The two are different physical properties of the same hardware tier and require different experimental designs.

The two timings. A global memory load has two physically distinct timings: the per-instruction *service time* (cycles between issue and value-available-to-consumer) and the *amortised cost per load* when many are in flight simultaneously. The amortised cost can be much smaller than the service time. The cycle-accurate CUDA benchmarking literature flags this directly: loads are “*variable latency instruction[s]*” and “*the GPU executes subsequent **independent** ops before the memory is actually loaded into the destination register*” [10] — a measurement that does not control for in-flight loads is timing a different physical event in each regime.

Why this matters for cache modifiers. A single transaction has nothing for a fill-and-eviction policy to act on; the PTX wording of `.cs` encodes this — it limits “*cache pollution by temporary streaming data*” [5, §8.7.8.1], which only applies in the many-loads regime. Empirical confirmation: modifiers collapse at $w = 1$ and the ranking opens only as warps add reuse pressure (Figure 4.5, Table 4.4). Modifier effects belong in the throughput regime, but reading the per-tier latency that policy acts against requires the latency regime too — the experiments must be designed in pairs.

Implications for design. The latency regime needs a design that serialises loads — Mei and Chu state the requirement: “*every single data access latency*” must be resolved “*in a kernel with a single thread and single CTA*” [18]. The throughput regime needs the opposite: many independent in-flight loads. Sections 3.3 and 3.4 describe each.

3.2 Clock Instrumentation

This section establishes the two on-chip timing primitives and quantifies their resolution, overhead, and bias — the ISA-mandated equality between them is what licenses the measurement (timing-side foundation for Claim 3).

How. Two primitives sampled in the same kernel. Primary: `%clock64`, “a predefined, read-only 64-bit unsigned cycle counter” that “wraps silently” [5, §10.24]. Cross-check: `%clock`; the ISA specifies “the lower 32-bits of `%clock64` are identical to `%clock`” [5], and across 120 trials at four WS sizes (1 KB–1 MB) the deltas agreed bit-for-bit, verifying that the CS2R lowering of `%clock64` is reading the architectural counter the ISA names rather than a re-clocked or aliased register.

The bracket pattern used for all throughput measurements is most precisely understood at the SASS level. After `ptxas` compilation with `-arch=sm_86`, the inner timed region disassembles to the SASS sequence below (annotation added; `cuobjdump` output, condensed for the `.ca` variant). The leading *prime* loop fills the cache to steady state at unmeasured cost; the timed bracket then captures the steady-state pass over a private buffer.

Listing 3.1: SASS-level disassembly of the `clock64` bracket pattern for global load throughput measurement on SM86, `.ca` variant. The first CS2R reads the 64-bit cycle counter into the timing register pair; the inner `LDG.E.STRONG.SM` stream is the timed event; the second CS2R closes the bracket. The prime loop runs once before this to fill the cache to steady state.

```

// --- prime loop (untimed) elided ---
CS2R    R6, SR_CLOCKLO    // t0 = lo(%clock64)
CS2R    R7, SR_CLOCKHI    // t0 = hi(%clock64)
// --- inner B-reload loop, REPS x B_LEN/32
//          iterations ---
LDG.E.STRONG.SM R10, [R4]  // .ca-modified load (one of 128 LDGs/warp)
LDG.E.STRONG.SM R11, [R4+0x80] // ... (coalesced over the warp's lane)
LDG.E.STRONG.SM R12, [R4+0x100] // ...
// [stride continues to B_LEN, then REPS]
CS2R    R8, SR_CLOCKLO    // t1 = lo(%clock64)
CS2R    R9, SR_CLOCKHI    // t1 = hi(%clock64)
IADD3   R6, R8, -R6, RZ    // delta lo
IADD3.X R7, R9, -R7, RZ, ... // delta hi (with carry)
// (t1 - t0) / REPS stored to cycles[warp_id]
```

For readers approaching this from CUDA C++ rather than SASS, the same bracket pattern is what the inline-PTX form below compiles to:

Listing 3.2: Inline-PTX C++ form of the same bracket pattern. `ptxas` lowers each `ld.global.ca` to one of the `LDG.E.STRONG.SM` instructions in the SASS listing above; the two `mov.u64 %%clock64` reads lower to the `CS2R SR_CLOCKLO/HI` pairs.

```
long long t0, t1;
asm volatile("mov.u64_0, %%clock64;" : "=l"(t0));
for (int r = 0; r < REPS; r++)
    for (int k = lane; k < B_LEN; k += 32)
        asm volatile("ld.global.ca.f32_0, [%1];"
                      : "=f"(v) : "l"(B+k) : "memory");
asm volatile("mov.u64_0, %%clock64;" : "=l"(t1));
cycles[warp_id] = (t1 - t0) / REPS;
```

What we found. Three measurement-confidence properties. (i) **Counter resolution.** Minimum delta between consecutive reads is 2 cy ($p_{95} = 2$ cy). (ii) **Instruction overhead.** The CS2R SASS read takes 2 cy on RTX 3060 Ti, p_{95} stable across 1 000 null iterations. (iii) **Measurement bias.** At the modifier-sweep operating point (16 KB-per-warp B-reload), the timer reads contribute 0.12% of the bracketed cycle count (vs. 4.35% in a 4-load scenario and 0.85% in a 1-load scenario, both shown for context). The 0.12% sits two orders of magnitude below the smallest cycle delta this study resolves (the $\sim 26\%$ sub-L1 `.nc/.ca` gap), licensing the modifier comparison to read directly off raw cycle counts without overhead correction.

What it means. `%%clock64` read pairs satisfy the ISA-mandated low-32-bit equality with `%%clock` at every measurement, so the cycle counts can be trusted. The SM frequency used for cycle $\rightarrow$ GB/s conversion (Section 4.3) is read at runtime as the SM clock rate (CUDA’s `cudaDeviceProp::clockRate`; 1.680 GHz on the RTX 3060 Ti, 1.545 GHz on the RTX 2080 Ti). All results below are reported from `%%clock64`.

3.3 The Latency-Regime Design: Pointer-Chase

This section describes the design that recovers the per-instruction service time at each cache tier (latency input to Claims 3 and 4).

Why. The throughput regime amortises many in-flight loads into one number that can be much smaller than the in-isolation service time. To recover the latter the experiment must guarantee no two loads are in flight at once: if any additional load issues before the first retires, the warp scheduler will overlap them. The instrumentation only counts cycles between reads, so the serialisation has to come from the kernel design — making

each load’s address depend on the previous load’s result, forcing load $N + 1$ to wait for load N to retire. This is what the pointer-chase achieves, and it is what makes its cycle delta interpretable as per-tier service time.

How. The pointer-chase, introduced by Wong et al. for GT200 [7] and applied across every subsequent NVIDIA generation. Wong describes the design directly: “*Our microbenchmark executes a sequence of pointer-chasing dependent reads to global memory*” [7], where each load’s address is the result of the previous load. The dependency forces serialisation — load $N + 1$ cannot issue until load N retires — so at most one load is in flight at any moment. The `clock64` bracket spans the entire chain; dividing by chain length gives the per-hop latency. Mei and Chu refined this for GPU clock counters: the fine-grained P-chase “*records and analyses every single data access latency in a kernel with a single thread and single CTA*” [18]. Jia et al. [6] apply the same construction on Volta and Turing; Yan et al. [9] extend it to Ampere.

A flush protocol is necessary to clear residual L2 warmth before DRAM-range measurements: a $2 \times L2$ streaming access (a contiguous array of $2 \times 12\text{CacheSize}$ bytes) evicts all lines from L2 before the timed pointer-chase begins. Chain sizes span 2 KB to 16 MB, bracketing the $L1 \rightarrow L2 \rightarrow \text{DRAM}$ transitions for both target GPUs.

What we found. Three latency tiers are clearly visible (results in Section 4.2); boundaries align with the device-reported L1 and L2 capacities to within one chain-size step.

What it means. The L2/L1 latency ratio sets the upper bound on what a tier-routing modifier can save; the measured throughput-regime ratios in Section 4.4 sit below this bound because pipelined loads amortise per-tier service time across many in-flight requests.

3.4 The Throughput-Regime Design: B-Reload Sweep

This section defines the *B-reload sweep*: each warp owns a private float buffer B of fixed size; B is primed once at unmeasured cost, then a $64\text{-rep} \times 128\text{-LDG}$ inner loop is timed across the steady-state cache. The throughput numbers feed Claim 4 (working-set-conditional performance map) and Claim 3 (cycle ratios that NCU cross-validates).

Why. Complementary to pointer-chase: many back-to-back independent loads keep the memory pipeline full. Modifier policy effects are observable only under sustained reuse pressure.

How. Working-set size is varied via co-resident warp count, following Wong et al. [7]. Each warp’s private buffer is 16 KB on SM86 (8 KB on SM75); aggregate WS grows by that step per added warp. L1 is exactly filled at eight co-resident warps on both architectures. Warp counts $\{1, 2, 4, 6, 8, 12, 16, 32\}$ span sub-L1 through L2-bound to DRAM-bound. Modifier choice and warp count are varied independently with all other factors held constant. All results are medians over 120 repetitions with 95% bootstrap CIs (10 000 resamples).

Validity threat: the warmup confound. `.ca` fills L1 on warmup; `.cg` never does. Their warmup curves are structurally different, so a benchmark that begins timing before steady state would record a warmup/steady-state blend belonging to neither regime. The threat is amplified at the instrumentation level: *“even if the load instruction is correctly placed between two `clock()` queries, the loads might not [have] finished when the second `clock()` is executed”* [10], so a cold L2 fill can bleed ~ 290 cycles into the next interval — enough to reverse modifier rankings. The prime phase (a full untimed pass over every warp’s buffer before timing begins) eliminates this. The `.ca/.cg` pair is the ISA-worst case for warmup contrast, and the measured cold-vs-steady-state advantage differs by only 0.64 pp (41.4% vs. 40.8%) — indistinguishable from noise.

Bandwidth conversion. Throughput is reported either as cycles per element or as GB/s. The conversion uses

$$\text{cy/elem} = \frac{\text{total cycles}}{\text{array length}}, \quad \text{GB/s} = \frac{4 \text{ B}}{\text{cy/elem} \div f_{\text{SM}}},$$

where f_{SM} is the runtime-queried SM clock rate (CUDA’s `cudaDeviceProp::clockRate`, §3.2). The L1/L2 boundary in the bandwidth sweep is determined at runtime from `cudaDeviceProp::l2CacheSize` rather than architectural documentation; this matters for SM75, whose L2 is 5.5 MB — not the 4 MB figure quoted in several prior microbenchmarking studies [6, 8].

Forward pointer. Section 3.5 converts each modifier’s PTX intent into a predicted clock-data signature on the B-reload sweep. Section 4.4 then tests those predictions quantitatively.

3.5 Cache Policy Semantics and Predicted Clock-Data Signatures

This section converts each PTX modifier intent into a specific clock-data signature the B-reload sweep should produce. Stating predictions up front turns each measured curve into a falsification opportunity for a specific hardware claim — the PTX ISA frames the four modifiers as performance hints rather than as guaranteed semantics (Section 2.2), so the only way to know what each does on real silicon is to convert the hint into an expected signature and check.

Per-modifier prediction. (Verbatim PTX intent in Section 2.2; this list states only the predicted B-reload signature.)

- .ca.** Cycles drop sharply at sub-L1 working sets as the second pass and beyond hit L1; cycles rise at L1 capacity as the working set spills out of L1; cycles plateau at L2 latency until L2 capacity, then rise again to DRAM.
- .cg.** Cycles flat at L2 service cost across all sub-L1 and L2-bound working sets (no L1 acceleration possible by ISA construction); convergence with the others at DRAM-bound. NCU should report *exactly* 0% L1 hit rate at every condition, because the structural L1 bypass is an architectural guarantee, not a probabilistic placement.
- .cs.** Indistinguishable from .ca below L1 (the line is filled in L1; the evict-first hint matters only when something else needs the slot); a measurable advantage in the L2-bound regime, where the evict-first hint at L2 prevents .cs-loaded lines from displacing reusable lines and reduces effective L2 service cost; convergence at DRAM-bound.
- .nc.** At sub-L1, an advantage over .ca if the read-only path’s bandwidth gain dominates its latency penalty (which the B-reload regime, with many loads in flight, is set up to expose); convergence at L2-bound and DRAM-bound where the path’s intrinsic bandwidth ceases to be the bottleneck. The phrase “*sufficient parallelism*” that the ISA uses to qualify the .nc advantage [5, §8.7.8.1] is left unquantified by NVIDIA — the gap this report closes empirically.

Per-tier latency baseline. The pointer-chase results (Section 4.2) yield $t_{L1} = 39$ cy and $t_{L2} = 329$ cy on SM86 — an $8.4\times$ raw ratio. The B-reload throughput regime amortises this across many in-flight loads, so the measured sub-L1 .cg/.ca ratio is smaller than $8.4\times$; Section 4.4 reports it at $2.35\times$.

3.6 SASS Validation

This section establishes that the modifier intent written in PTX survives compilation to SASS as a distinct opcode on each architecture — the four modifiers must be distinct at the instruction level rather than synonymous PTX aliases (supports Claims 1 and 3).

Why. PTX is virtual; `ptxas` can silently remap a modifier, and “*the compiler can reorder instructions, so we do not exactly know the exact set of source code lines we measure*” [10]. Cycle-accurate timing requires SASS-level verification.

How. Two concerns shape the procedure. (i) The warp scheduler may issue a clock-counter read while a load is still in flight (no register dependency on the load result). (ii) Global loads are variable-latency: “*the GPU executes subsequent **independent** ops before the memory is actually loaded into the destination register*” [10], so without an explicit data dependency the closing timer can fire before the load retires. Both are addressed by the *first-consumer* pattern: an instruction reading the load destination register is interposed between the LDG and the second timer read; SM86 stalls on register RAW hazards [13], so the first consumer cannot issue until the load data has arrived. The principle is stated in the cycle-counter benchmarking literature: “*If an upcoming instruction depends on the destination register of [the load], then, the programmer should explicitly wait on the barrier set by [the load] instruction*” [10].

The SASS excerpt below, disassembled from the project’s `ldg_first_consumer_kernel`, illustrates the pattern:

Listing 3.3: SASS first-consumer timing pattern for LDG latency measurement (SM86). CS2R reads `SR_CLOCKLO` (%clock). The FADD at `0x80` is the first consumer of R2 (the LDG destination); it stalls until the global load completes, so the second CS2R at `0x90` captures the post-load cycle count.

```
/*0060*/ CS2R R6, SR_CLOCKLO      // t0 = clock32
/*0070*/ LDG.E.STRONG.SM R2, [...] // issue LDG (.ca); result -> R2
/*0080*/ FADD R13, R2, 1          // FIRST CONSUMER: reads R2 -> stall until LDG done
/*0090*/ CS2R R8, SR_CLOCKLO      // t1 = clock32 (after load has completed)
/*00b0*/ IADD3 R10, -R6, R8, RZ // delta = t1 - t0
```

All four PTX cache modifiers and the Triton kernel load instructions were validated by:

1. Compiling reference kernels with `nvcc -arch=sm_86` and disassembling with `cuobjdump --dump-sass`, the recommended disassembly route in prior cycle-counter benchmarking work: “*the .cubin file can be disassembled to human-readable format using*

CuAssembler” [10, 1]. `cuobjdump` is the primary tool here, with *CuAssembler* used for instruction-field decoding.

2. Decoding the cache modifier field (bits [15:12] of instruction `qword2`) and matching it against the encoding table in Appendix B.
3. Binary-decoding Triton-compiled gather/GeLU/softmax kernels to determine the modifier field actually emitted.

What we found. Each PTX modifier compiles to a distinct SASS opcode (Table 2.2). The Triton gather kernel decodes at offset `0x00e0` as `qword2 = 0x000ea2000c1e9900`; bits [15:12] = `0x9` = `LDG.E.CONSTANT`, which is the SASS form of `.nc`.

What it means. The four PTX modifiers are not nominal aliases that collapse in compilation; they are four distinct hardware instruction paths. Triton’s elementwise kernels lower to the read-only texture-cache path regardless of the PTX modifier requested in the source — a result that drives the choice-locked dead-zone analysis in Section 4.5.

3.7 NCU as the Independent Confirmation Instrument

This section pre-stages the role NCU plays in the validation chain (second half of Claim 3): `clock64` cannot directly report which cache tier served any individual load, so an instrument operating on a different physical principle is required.

Why. `clock64` measures cycles between instruction boundaries. A measured cycle count of 39 cy is consistent with an L1 hit, but it is also consistent with any other event that happens to take 39 cycles. The cycle count alone does not prove L1 placement; it is one observable, and proof requires a second observable that cannot be confused with the first.

How. NCU’s `l1tex__t_sector_hit_rate.pct` fires at the tag-lookup stage of the L1TEX unit (which handles “*global, local, shared, texture, and surface memory reads and writes*” [11]), independent of in-kernel timing. A sector is an “*aligned 32-byte chunk of memory in a cache line or device memory*” [11], and a sector access counts as a hit only “*if the tag is present and the sector-data is present within the cache line*”, with both “*tag-misses and tag-hit-data-misses... classified as misses*” [11]. NCU reports *architectural routing*; `clock64` reports *timing*; agreement at tier boundaries licenses both.

What this enables. A specific failure criterion, framed as *tier disagreement*: if `clock64` indicates L2-tier service while NCU simultaneously reports a high L1 hit rate (or vice versa), the timing instrument has failed. Section 4.1 reports the result — no tier disagreement at any condition tested, plus a single within-tier cycle-vs-hit-rate disagreement at $w = 6$ that turns out to be positive evidence (L1 bank-stall) rather than instrumentation failure. The aggregate-vs-per-load bound on this validation is discussed in Section 5.4.

Chapter 4: Results

This chapter reports the experimental results in the order they relate. Each section opens with the question it answers, names the figures and tables that hold the data, reports the numbers, and ends with what the result proves. The five results, in order: cross-instrument validation of `clock64` against NCU on cache tier (Section 4.1, hypothesis H); per-tier latency on both GPUs (Section 4.2, baseline for F1); per-tier throughput on both GPUs (Section 4.3, baseline for F1); the four-modifier working-set-conditional performance map (Section 4.4, F1); the Triton dead zone vs. hand-written CUDA actionable window (Section 4.5, F2).

4.1 `clock64` Validation Against NCU

Claim Advanced by This Section

Hypothesis H. `clock64` alone cannot prove which cache tier served a load — two configurations with the same cycle count could in principle be reaching different tiers if the pipeline depth differs. The tier becomes unambiguously identifiable when `clock64` is paired with NCU hardware counters: NCU’s L1 hit rate is reported on a per-instruction-class basis from the L1TEX tag arrays, so a 0.00% L1 hit reading under `.cg` *architecturally* confirms L1 bypass independently of timing, and a 98.46% L1 hit reading under `.ca` confirms L1 service. Once NCU has assigned the tier, the corresponding `clock64` cycle count is anchored to a known cache tier, and any subsequent change in cycles under the same NCU classification (e.g., the bank-stall regime at $w = 6$) is attributable to a within-tier microarchitectural effect rather than a tier change.

Question. Does `clock64` agree with NCU on cache tier at every working-set point, and where they disagree, is the disagreement explainable as a real microarchitectural effect?

Data. Table 4.1 (SM86 cross-instrument record), Figure 4.1 (dual-axis view for both GPUs).

Why a separate validation step is required. A cycle count is consistent with multiple physical events (L1 hit, L2 hit on a deeper pipeline, stall on a serviced access). NCU inspects L1TEX tag arrays directly and identifies the tier architecturally rather than by

timing; the validation tests whether the two agree, with disagreement attributed to a real microarchitectural effect rather than instrumentation error. Two independent checks: (1) NCU confirms L1 bypass for `.cg` and L1 fill for `.ca`; (2) at $w = 6$ the cycle counter detects an L1 bank-stall the hit-rate counter is structurally blind to.

Evidence 1: NCU independently confirms L1 bypass and L1 fill. The NVIDIA *PTX ISA Reference Manual* (Release 8.5, §8.7.8.1, “Cache Operators”) [5] specifies the architectural intent of the two modifiers being validated here. It states that `.cg` caches “*at global level (cache in L2 and below, not L1)*” [5, §8.7.8.1], and that `.ca` “*cache[s] at all levels, likely to be accessed again*” [5, §8.7.8.1]. These are the official architectural definitions; the validation in the next paragraphs checks whether NCU and `clock64` both observe behaviour consistent with them. NCU verifies both with no dependence on timing. At every sub-L1 warp count (using w for the active-warp count, the same independent variable defined in Section 3.4: $w \in \{1, 2, 4\}$ on SM86, working set ≤ 64 KB), NCU reports L1 hit rate of 98.46% under `.ca` and 0.00% under `.cg` (Table 4.1). The 0.00% is not sampling noise: Section 3.6 confirmed `.cg` lowers to LDG.E.STRONG.GPU, whose GPU-scope coherence domain structurally excludes L1D, so no L1 sector is ever allocated by this instruction.

Evidence 2: tier-level cycle counts agree across the working-set sweep. Table 4.1 records the cross-instrument values at every condition. At sub-L1 ($w = 1$, WS = 16 KB) `.ca` is L1-served and `.cg` is L2-served; the cycle ratio ($2.35\times$) reflects the per-tier service-time difference attenuated by pipeline overlap. At the L2 boundary ($w = 16$, WS = 256 KB) NCU reports 1.13% `.ca` and 0.01% `.cg` L1 hits (both L2-served by capacity), with `clock64` agreeing on tier to within 0.3%. Post-L2-saturation ($w = 32$, WS = 512 KB) both instruments register *complete collapse*: `clock64` ratio = 1.000, NCU `.ca` L1 hit = 0.02%, NCU `.ca` L2 hit = 98.36% — the working set exceeds L1 by capacity but stays inside L2, so all four modifiers route through identical paths through the saturated L2 line-replacement bandwidth. The hit-rate figures interpret via the L1TEX sector classification (defined in Section 3.7: hit only if “*the tag is present and the sector-data is present*” [11]).

Evidence 3: `clock64` reveals sub-structure invisible to hit-rate counters. At $w = 6$ (WS = 96 KB), NCU reports `.ca` L1 hit rate of 98.46% — identical to $w = 1$ through $w = 4$, as expected since the working set still fits in the 128 KB L1. Yet `clock64` records `.ca` = 2290 cy, a 34% increase over the 1711 cy at $w = 4$, while `.cg` cycles increase only 5% (4019 $\rightarrow$ 4234 cy). Because NCU confirms L1 hits are unchanged, the slowdown cannot be from misses. The mechanism is L1 bank contention: six concurrent warps

issuing simultaneous L1 reads serialise on shared cache banks, adding stall cycles to served accesses without producing any miss event. NCU’s own warp-stall taxonomy is consistent: a warp stalled on a serviced L1 access appears as a “*memory dependency (result of memory instruction)*” stall rather than a miss event [11], so a bank-stall increase is invisible to hit-rate counters by design. This is positive evidence for the `clock64` methodology, not a failure: the timing instrument detects a real microarchitectural effect that the hit-rate counter is structurally blind to.

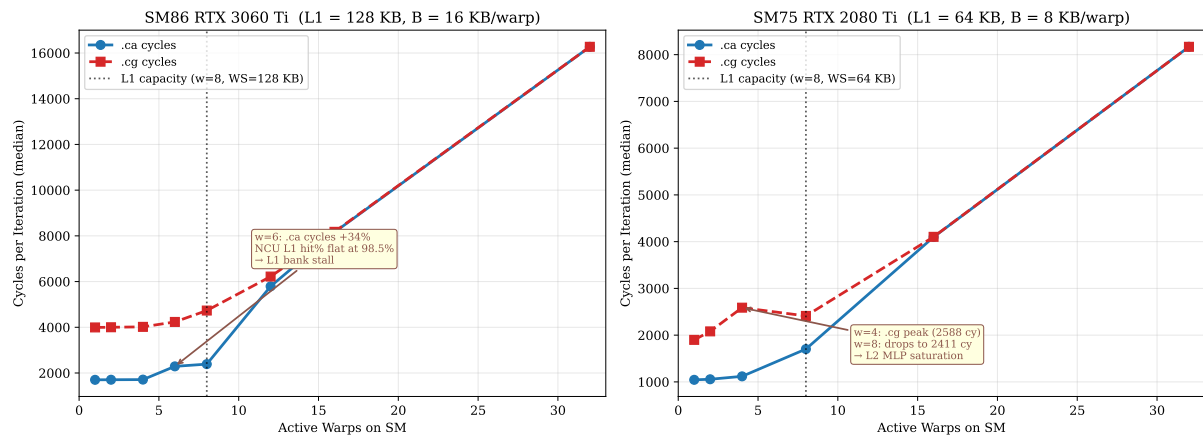


Figure 4.1: Dual-axis cross-instrument view for both GPUs. Left axis: `clock64` cycles per iteration for `.ca` (filled circles) and `.cg` (squares). Right axis: NCU L1 hit rate for `.ca` (triangles, dashed). **Read it like this:** on SM86 (left), the `.ca` cycle trace spikes 34% at $w = 6$ while the NCU hit-rate trace stays flat at 98.5% — the bank-stall signature visible simultaneously in both instruments. SM75 (right): the L1 saturation collapse at $w = 8$ (WS = 64 KB = L1 capacity) is marked.

Figure 4.1 shows the dual-axis cross-instrument view for both GPUs. On SM86 the $w = 6$ bank-stall signature is directly visible as the cycle trace spiking while the hit-rate trace stays flat at 98.5%. On SM75 the L1 saturation collapses more sharply: from 98.46% at $w = 4$ (WS = 32 KB) to 75.07% at $w = 8$ (WS = 64 KB) and near-zero at $w = 12$, consistent with the smaller 64 KB L1. Table 4.1 gives the complete cross-instrument record for SM86; the SM75 record agrees in structure (sub-L1 NCU `.ca` hit 98.45–98.46%; `.cg` hit 0.00%; L2-bound convergence within 0.2%).

What this proves. Tier-level agreement at every tested condition (H); the cycle counter additionally detects a 34% bank-stall at $w = 6$ invisible to the hit-rate counter. Both instruments report warp-level averages, licensing agreement on conditions not individual loads (Section 5.4).

Table 4.1: Cross-instrument validation: NCU hardware counters vs. `clock64` for SM86 (RTX 3060 Ti). NCU `.cg` L1 hit is 0.00–0.01% at all conditions, confirming structural L1 bypass architecturally. † At $w = 6$, NCU L1 hit rate is unchanged from $w = 4$ yet `clock64` records a 34% `.ca` cycle increase: bank-stall latency detected by timing but invisible to hit-rate counters.

Warps	WS (KB)	NCU <code>.ca</code> hit	NCU <code>.cg</code> hit	<code>.ca</code> cy	<code>.cg</code> cy	Ratio
1	16	98.46%	0.00%	1 701	3 994	2.348
2	32	98.46%	0.00%	1 705	4 001	2.346
4	64	98.46%	0.00%	1 711	4 019	2.349
6 [†]	96	98.46%	0.01%	2 290	4 234	1.849
8	128	78.70%	0.01%	2 388	4 732	1.982
12	192	13.32%	0.01%	5 792	6 219	1.074
16	256	1.13%	0.01%	8 158	8 176	1.002
32	512	0.02%	0.01%	16 270	16 272	1.000

4.2 Hierarchy Latency: Per-LDG Service Time

Question. What is the per-instruction service time of an isolated LDG on each tier?

Data. Figure 4.2, Table 4.2.

Result.

- **SM86:** $t_{L1} = 39$ cy, $t_{L2} = 329$ – 333 cy, $t_{DRAM} = 533$ – 547 cy — L2/L1 ratio $8.4\times$.
- **SM75:** $t_{L1} = 40$ cy, $t_{L2} = 232$ – 238 cy, $t_{DRAM} = 358$ – 432 cy — L2/L1 ratio $5.9\times$.
- The 128 KB chain on SM86 straddles the L1 capacity (mixed L1/L2 latency); shown in pink in Figure 4.2 and excluded from the per-tier summary. True L2 latency read at 512 KB.

Table 4.2: Pointer-chase latency per memory tier (corrected, mixed boundary excluded).

GPU	L1 latency	L2 latency	DRAM latency	L2/L1 ratio
RTX 3060 Ti (SM86)	39 cy	329–333 cy	533–547 cy	$8.4\times$
RTX 2080 Ti (SM75)	40 cy	232–238 cy	358–432 cy	$5.9\times$

What this proves. Three latency tiers resolved on each architecture; L2/L1 ratios set the upper bound on what any tier-routing modifier can save (Claim 2). Yan et al. [9] report 33/200/290 cy on the larger-die GA100 (A100); same tier ordering at higher absolute values here on the smaller-die GA104 confirms the instrumentation is on the correct scale.

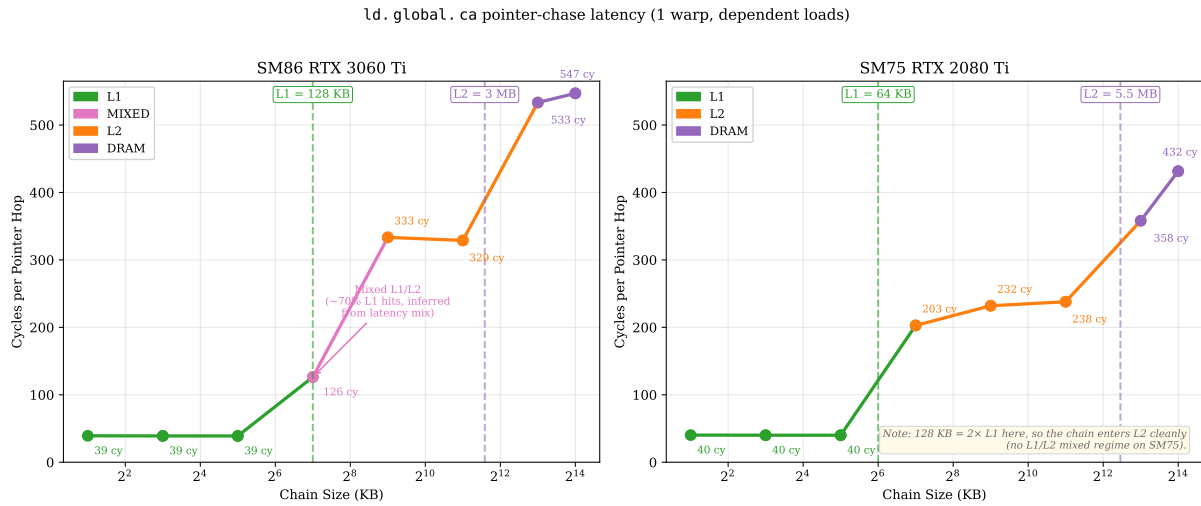


Figure 4.2: Pointer-chase latency in cycles per hop vs. chain size. The reclassified mixed L1/L2 boundary point at 128 KB (SM86) is shown in pink. True L2 latency (333 cy) is measured at 512 KB, where the chain cleanly resides in L2. SM75 (right) shows a less pronounced boundary effect because 128 KB is 2× the SM75 L1 capacity.

4.3 Hierarchy Throughput: Amortised Per-Element Cost

Question. What amortised per-element cost (and GB/s) does a single warp sustain in the pipelined-streaming regime on each tier?

Data. Figure 4.3, Figure 4.4, Table 4.3.

Why. Substrate for the four-modifier sweep (Section 4.4). Complementary to Section 4.2: B-reload streams through a contiguous float array with no inter-load dependencies, so the warp scheduler pipelines LDGs freely — the regime where modifier policy effects are observable under sustained reuse pressure.

Result. Three throughput plateaus on each architecture mark the L1, L2, and DRAM regimes.

- **SM86 under .ca.** L1 tier ($WS \leq 64$ KB): 0.387 cy/elem, 17.4 GB/s. L2 tier (128 KB–2 MB): ≈ 0.663 cy/elem, 10.1 GB/s. DRAM tier ($WS \geq 4$ MB): ≈ 1.38 cy/elem, 4.9 GB/s.
- **Tier transitions.** 128 KB transition matches SM86 L1 capacity; the ≈ 4 MB transition lies above the 3 MB L2 (set-associativity effect).
- **Cross-architecture.** SM75 same structure at lower throughput (Table 4.3); same modifier ordering on both architectures.

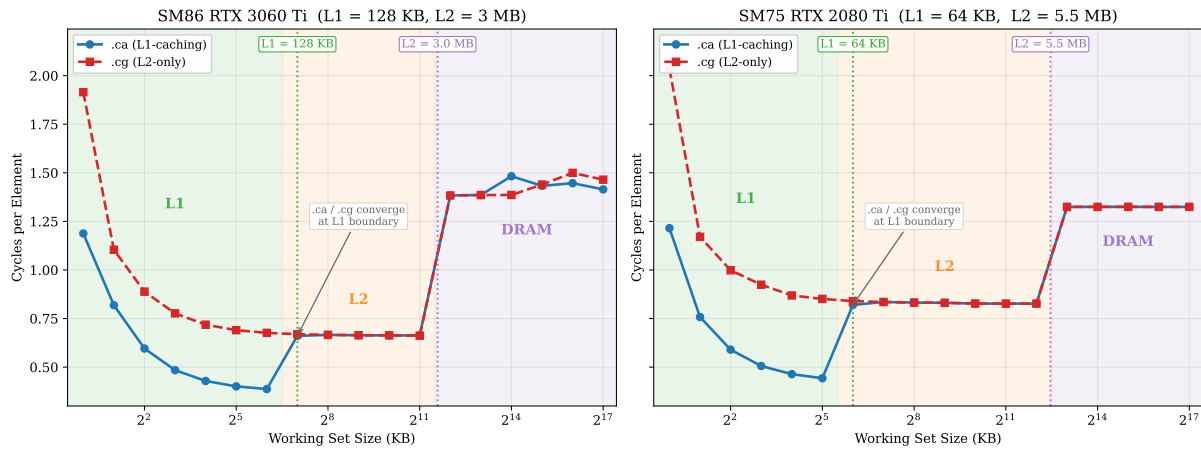


Figure 4.3: Hierarchy throughput: cycles per element vs. working-set size for `.ca` and `.cg` global loads in the independent-load (pipelined) regime. Background shading indicates the detected tier. Left: SM86 (L1=128 KB, L2=3 MB). Right: SM75 (L1=64 KB, L2=5.5 MB). **Read it like this:** three distinct plateaus per panel mark the L1, L2, and DRAM service regimes.

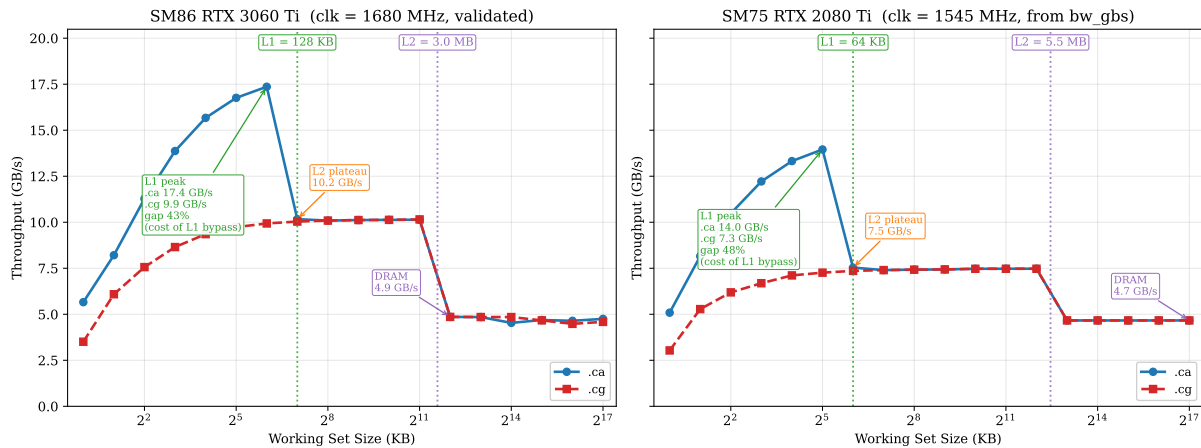


Figure 4.4: Achieved throughput in GB/s vs. working-set size for both GPUs. **Read it like this:** peak L1 throughput under `.ca` is substantially higher than under `.cg`; the two converge at the L2 and DRAM plateaus where the policy distinction stops mattering.

Table 4.3: Peak throughput per memory tier for both GPU platforms (validated clocks).

GPU	Tier	.ca peak (GB/s)	.cg peak (GB/s)	.ca/.cg ratio	Transition WS
RTX 3060 Ti (SM86)	L1	17.4	9.9	$1.76\times$	128 KB
	L2	10.2	10.2	1.00	3 MB
	DRAM	4.9	4.9	1.00	—
RTX 2080 Ti (SM75)	L1	14.0	7.3	$1.92\times$	64 KB
	L2	7.5	7.5	1.00	5.5 MB
	DRAM	4.7	4.7	1.00	—

What this proves. Three throughput tiers resolved with sharp L1/L2 boundary transitions (input to F1). Sub-cycle per-element figures are pipelined throughput, not per-LDG service time — each LDG services 32 coalesced lanes with many in flight.

4.4 Modifier Effects in the Throughput Regime

Claim Advanced by This Section

Claim 4. With instrumentation validated, the four modifiers produce a working-set-conditional performance map: different rankings at sub-L1, L2-bound, and DRAM-bound regimes, with secondary effects (L1 bank contention, L2 saturation) visible to `clock64`.

Question. How do the four modifiers compare on each tier, and how does the comparison change as WS crosses tier boundaries?

Data. Figure 4.5 (.ca vs .cg); Figure 4.6 (all four); Figure 4.7 (sub-L1 and L2-bound rankings); Table 4.4 (numerical summary).

Result 1: two-modifier collapse (Figure 4.5). Sub-L1 ($w < 16$): .ca stable at ≈ 1701 cy (SM86) / ≈ 1044 cy (SM75); .cg at $2.35\times$ higher (L2 service). L2-bound ($w \geq 16$): the two converge to within 0.2%. Bootstrap 95% CI at sub-L1 is a point estimate (± 0 cy, 120 trials). The collapse falsifies the naive reading “.ca selects L1, .cg selects L2”: modifiers specify fill/eviction *policy*; tier-of-service depends on residency.

4.4.1 Full Four-Modifier Taxonomy

Result 2: three regimes, distinct modifier orderings in each. With all four modifiers swept (Figure 4.6, Figure 4.7, Table 4.4), the regime structure becomes more detailed than the two-modifier collapse alone shows. Each regime gets its own paragraph below.

Sub-L1 regime ($w = 1$, WS = 16 KB on SM86; WS fits inside L1). .nc and .cs tie at 1259 cy (SM86) / 763 cy (SM75), both 26% faster than .ca; .cg is $2.35\times$ slower (pure L2 service forced by policy bypass). The read-only texture path (.nc) and Evict-First L1 path (.cs) carry lower effective latency than the **STRONG.SM** path used by .ca under reuse (matching Section 3.5). Bootstrap 95% CI at $w = 1$ is a point estimate (± 0 cy across 120 trials).

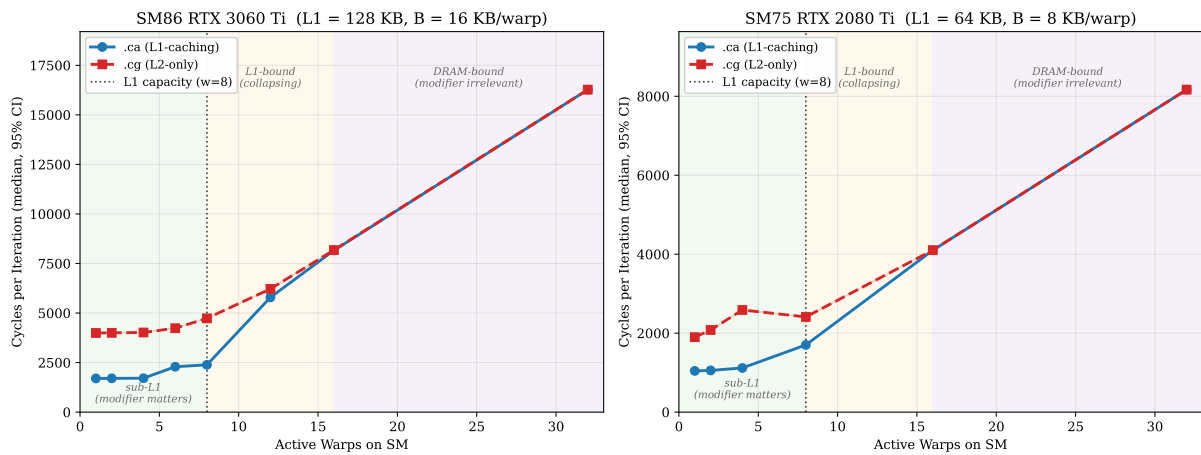


Figure 4.5: Two-modifier throughput collapse: `.ca` vs. `.cg` cycle counts as a function of active warp count. Bootstrap 95% CI shown (shaded). **Read it like this:** sub-L1, the two diverge by 2.35×; at the L1 capacity boundary they collapse to indistinguishable. SM86 (left): the $w = 6$ banking-stall anomaly is annotated. SM75 (right): same collapse pattern at $w = 16$ (WS = 128 KB > 64 KB L1).

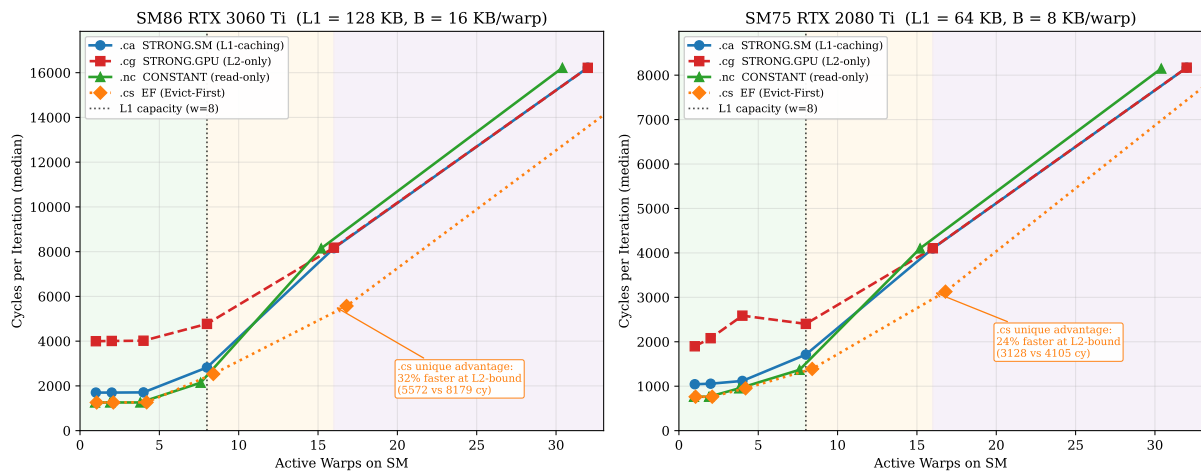


Figure 4.6: All four cache modifiers across warp counts for SM86 (left) and SM75 (right). **Read it like this:** `.cs` (Evict-First) is uniquely visible at L2-bound working sets, where it retains a $\approx 32\%$ cycle-count advantage over the other three.

Table 4.4: Four-modifier performance summary for SM86 (RTX 3060 Ti) and SM75 (RTX 2080 Ti).

Modifier	Sub-L1 ($w = 1$)			L2-bound ($w = 16$)		
	SM86 (cy)	SM75 (cy)	Ratio (SM86)	SM86 (cy)	SM75 (cy)	Ratio (SM86)
<code>.ca</code>	1701	1044	1.000	8154	4100	1.000
<code>.cg</code>	3998	1899	2.350	8179	4105	1.003
<code>.nc</code>	1259	763	0.740	8148	4101	0.999
<code>.cs</code>	1259	763	0.740	5572	3128	0.683

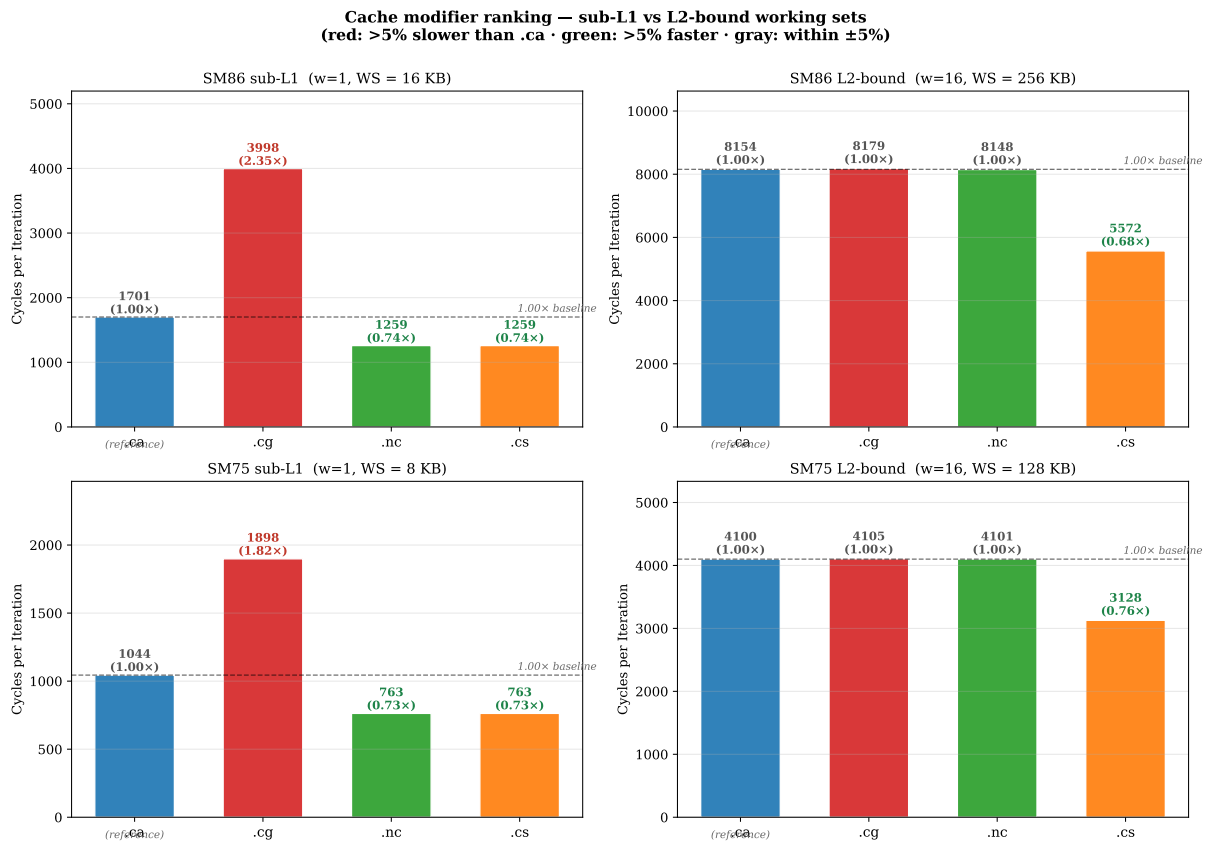


Figure 4.7: Modifier performance ranking at sub-L1 (top row, $w = 1$) and L2-bound (bottom row, $w = 16$) working sets for both SM86 (left column) and SM75 (right column). Values labelled as absolute cycles and ratio relative to .ca. Red ratio > 1 means slower than .ca; green < 1 means faster.

L2-bound regime ($w = 16$, $WS = 256$ KB; WS has spilled to $L2$). $.ca$, $.cg$, $.nc$ converge to within 0.3%; $.cs$ retains 5572 cy on SM86, 32% faster than the converged group. The PTX ISA specifies $.cs$ allocates “global lines with evict-first policy in $L1$ and $L2$ to limit cache pollution by temporary streaming data that may be accessed once or twice” [5]; the CUDA C++ Programming Guide reinforces this at the $L2$ level — “Memory accesses that occur with the streaming property are less likely to persist in the $L2$ cache because these accesses are preferentially evicted” [13]. By preferentially evicting just-loaded lines from both tiers, $.cs$ frees bandwidth for lines that will be reused. The 32% advantage is the first published quantification of this effect at the LDG instruction level.

Post-L2-saturation regime ($w = 32$, $WS = 512$ KB). All four modifiers converge to within 0.5%. The 512 KB working set sits well inside the $L2$ capacity on both GPUs (3 MB SM86, 5.5 MB SM75), but throughput nevertheless reaches DRAM-tier rates because the cyclic B-reload steady state evicts lines faster than the policy can retain them; the bottleneck is $L2$ line-replacement bandwidth rather than DRAM-bus saturation, and all four modifiers route through identical paths.

4.4.2 Secondary Effect: L2 Bandwidth Saturation

At sub- $L1$, $.cg$ cycle counts rise with warp count even though each warp’s buffer fits in $L1$ (Table 4.1): $.cg$ draws from the $L2$ bandwidth shared across co-resident warps, so adding warps saturates the shared $L2$ bus. $.ca$ ’s data stays in $L1$ and is stable. The $.cg/.ca$ ratio rises from 1.82 ($w = 1$) to 2.31 ($w = 4$) on SM75 — $L2$ contention, not a tier change. This motivates fixing the realistic-kernel grid to one SM (Section 4.5.3).

4.4.3 Cross-Architecture Comparison

What we found. Figure 4.8, Table 4.5. *Constant across architectures:* (i) three-tier hierarchy; (ii) sub- $L1$ ranking $.nc = .cs > .ca \gg .cg$; (iii) $.cs$ $L2$ -bound advantage (32% SM86, 24% SM75). *Changed:* SM86 has $2\times$ $L1$ capacity (128 vs. 64 KB), shifting the collapse point; SM86 has $1.4\times$ higher $L2$ latency (329 vs. 238 cy), raising the $L2/L1$ ratio from $5.9\times$ to $8.4\times$ and amplifying the $.cg$ penalty.

What this proves. Working-set-conditional performance map (Claim 4): three qualitatively distinct modifier rankings at sub- $L1$, $L2$ -bound, and post- $L2$ -saturation regimes. Secondary effects visible to `clock64` ($L1$ bank contention at $w = 6$, $L2$ bandwidth saturation under $.cg$) appear as cycle features the hit-rate counter is blind to. The qualitative ranking is preserved across SM86 and SM75; quantitative magnitudes track device-level

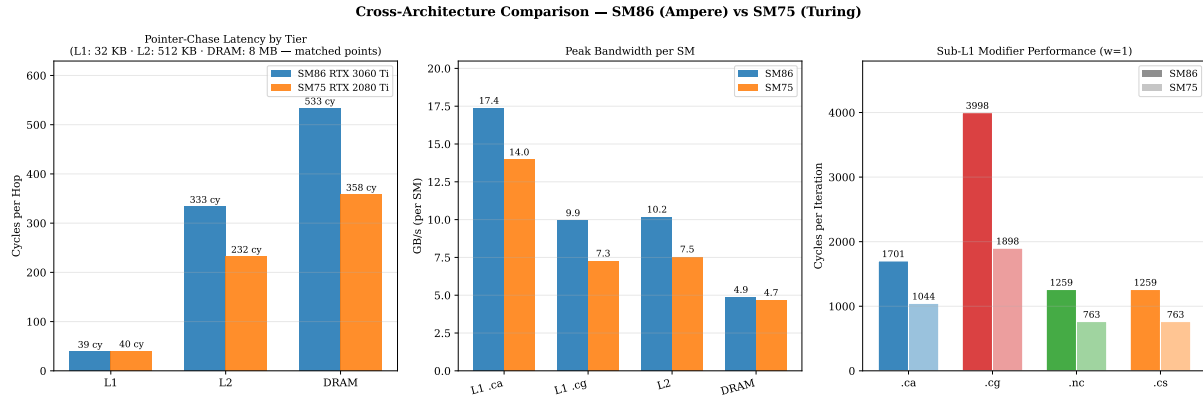


Figure 4.8: Cross-architecture comparison of latency, bandwidth, and sub-L1 modifier performance between SM86 (Ampere) and SM75 (Turing). **Read it like this:** the qualitative ranking is identical across architectures; SM86 shows higher L1 bandwidth but higher L2 latency.

Table 4.5: Cross-architecture summary of key measured quantities.

Metric	SM86 RTX 3060 Ti	SM75 RTX 2080 Ti
L1 latency	39 cy	40 cy
L2 latency	329 cy	238 cy
DRAM latency	533 cy	431 cy
L1 bandwidth (.ca)	17.4 GB/s	14.0 GB/s
L2 bandwidth	10.1 GB/s	7.5 GB/s
DRAM bandwidth	4.9 GB/s	4.7 GB/s
L1 capacity	128 KB	64 KB
L2 capacity	3 MB	5.5 MB
Sub-L1 .nc/.ca speedup	26%	27%
L2-bound .cs advantage	32%	24%

parameters (L2/L1 ratio, L1 capacity).

4.5 Triton vs. Hand-Written CUDA: Dead Zone vs. Actionable Window

Claim Advanced by This Section

Claim 5. The performance map is structurally inaccessible to Triton kernels but fully actionable in hand-written CUDA. The dead zone is a property of the Triton compilation path, not of the GPU hardware.

Question. Is the modifier map of Section 4.4 reachable from Triton kernels and from hand-written CUDA?

Data. Table 4.6, Table 4.7, Figure 4.9.

4.5.1 Triton Matmul and Attention: Structural Dead Zone

Triton matmul and attention on SM80+ emit `LDGSTS` — the SASS form of PTX `cp.async`, defined as “a non-blocking instruction which initiates an asynchronous copy operation” in which “cache operators” are “treated as performance hints only” [5]. The instruction routes data asynchronously from global memory to shared memory, bypassing L1D and the register file: the PTX cache modifier field is structurally inapplicable, since there is no LDG for it to attach to. No runtime benefit is achievable by changing it.

4.5.2 Triton Elementwise Kernels: Choice-Locked Dead Zone

For elementwise kernels (GeLU, Softmax, RMS-Norm), Triton maps one thread block per row, so the per-SM working set equals `hidden_size` $\times$ 4 bytes. Table 4.6 shows this is sub-L1 for every practical model configuration, on both architectures.

Binary decode of Triton-compiled kernels confirms `LDG.E.CONSTANT (.nc)` for gather, GeLU, and softmax kernels (`qword2 = 0x000ea2000c1e9900`, bits [15:12] = `0x9`). From the modifier map in Table 4.4, `.nc` is the optimal modifier at sub-L1 working sets. Any manual override to `.ca` would incur a 26% penalty; to `.cg`, a 135% penalty.

Table 4.6: Per-SM working set for Triton elementwise kernels on representative models. All configurations are sub-L1 for both SM86 and SM75.

Model	Hidden size	WS per SM (KB)	SM86 L1 (128 KB)	SM75 L1 (64 KB)	Modifier emitted
BERT-base	768	3.0	Sub-L1	Sub-L1	.nc
BERT-large	1024	4.0	Sub-L1	Sub-L1	.nc
GPT-2 small	768	3.0	Sub-L1	Sub-L1	.nc
LLaMA-7B	4096	16.0	Sub-L1	Sub-L1	.nc
LLaMA-13B	5120	20.0	Sub-L1	Sub-L1	.nc
LLaMA-70B	8192	32.0	Sub-L1	Sub-L1	.nc
GPT-4 class	12288	48.0	Sub-L1	Sub-L1	.nc

4.5.3 Hand-Written CUDA Kernels: Modifier Selection Is Actionable

Three realistic hand-written CUDA kernel patterns benchmarked across all four modifiers via PTX inline assembly. Method: 120 trials per (kernel, modifier) pair, median reported, grid fixed to one SM to hold L2 contention constant (Section 4.4.2). Patterns: 16 MB DRAM-bound streaming scan; 16 KB sub-L1 temporal-reuse loop; 1 M-index random-gather into a 4 MB array (L2-bound, irregular).

What we found. Table 4.7.

Table 4.7: Cache modifier effect on three realistic CUDA kernel patterns (SM86 RTX 3060 Ti). All values are relative to the .ca baseline. Best modifier per pattern shown in **bold**.

Pattern	.ca	.cg	.nc	.cs
Streaming scan (16 MB, DRAM-bound)	354.8 GB/s	−0.1%	−0.1%	+38.4%
Temporal reuse (16 KB buf, sub-L1)	19.9 GB/s	−52.2%	±0%	±0%
Random gather (1 M idx, 4 MB, L2-bound)	41.2 GB/s	+0.7%	−0.7%	−11.2%

What it means. The streaming result amplifies the microbenchmark prediction: the .cs Evict-First advantage extends to a natural DRAM-bound kernel. The temporal-reuse result confirms the .cg penalty is not a tight-loop artefact: routing every access through L2 when all data fits in L1 costs the full L2/L1 service-time difference. The random-gather row bounds the .cs result — evict-first prematurely discards lines that get reused, costing −11.2%. The dead zone is a property of how Triton lowers code, not of the modifiers themselves.

4.5.4 Modifier Selection Decision Guide

Figure 4.9 synthesises the findings into a practical decision tree. The optimal modifier depends on the access pattern and working-set regime.

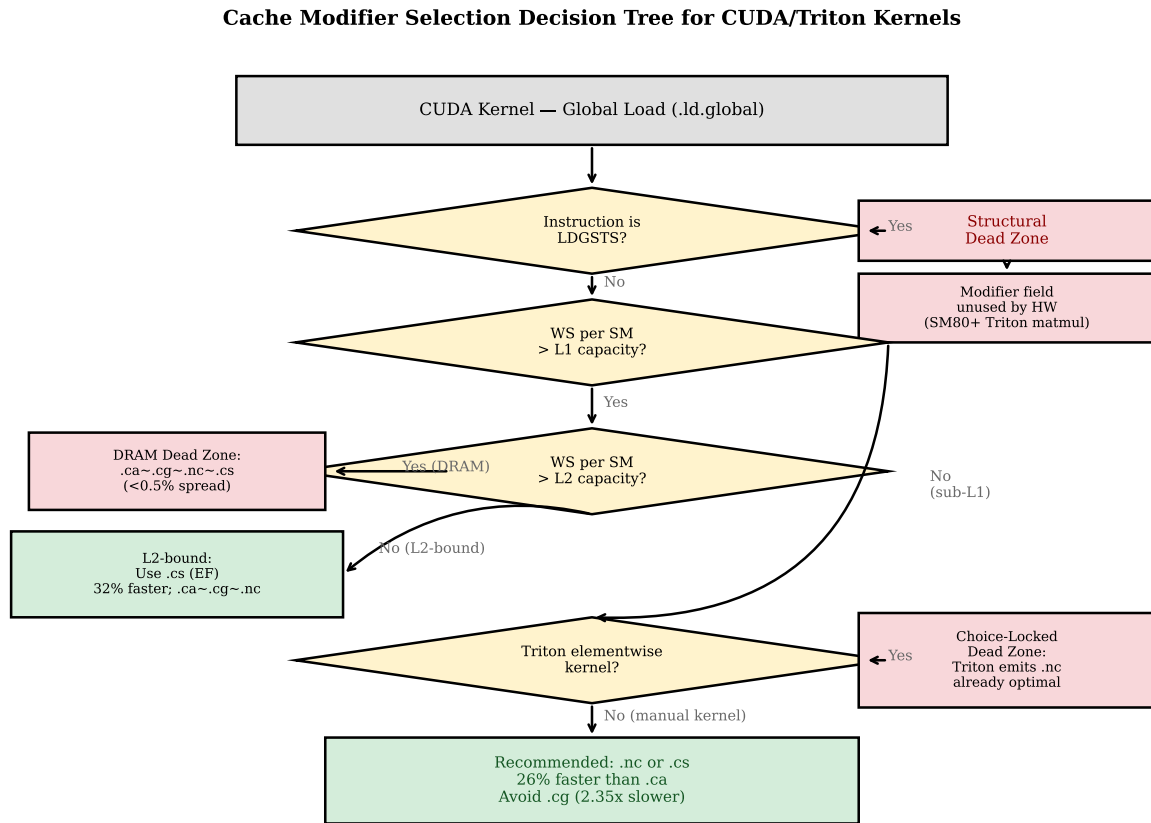


Figure 4.9: Cache modifier selection decision tree for CUDA and Triton kernels, synthesising all experimental results. The tree routes through: (1) instruction type check (LDGSTS vs. LDG), (2) working-set size relative to L1/L2 capacities, and (3) whether the kernel is a Triton elementwise kernel. All Triton paths reach a dead-zone outcome; hand-written CUDA paths reach the actionable region.

- **Sub-L1 manual kernels (temporal reuse):** use `.nc` or `.cs`; never `.cg` (52% penalty confirmed on a realistic 16 KB kernel).
- **DRAM-bound streaming kernels:** use `.cs` (+38.4% over `.ca` on a realistic 16 MB scan).
- **L2-bound with sequential access:** use `.cs` (32% faster in the microbenchmark);

degraded under random access (-11.2%).

- **DRAM-bound random access:** all modifiers converge; modifier choice irrelevant.
- **Any Triton kernel:** dead zone — modifier change either has no effect (structural) or hurts performance (choice-locked).

Chapter 5: Discussion

The experimental results raise three questions:

1. *Is the methodology valid?* (Section 5.1)
2. *What do the modifier effects say about hardware?* (Section 5.2)
3. *Why doesn't Triton expose them?* (Section 5.3)

Section 5.4 states the bounds within which the answers hold.

5.1 Is the Methodology Valid?

Yes: cross-instrument validation closes at every condition tested (Section 4.1 and table 4.1). Instrument bias (2-cy resolution + 2-cy overhead $\approx 0.12\%$) is two orders below the 26% effects of interest. The $w = 6$ banking stall is positive evidence: NCU and `clock64` measure different observables (tag-array occupancy vs. service time), so a serialised L1 bank access inflates cycles without a miss event.

5.2 What Do the Modifier Effects Say About Hardware?

Three observations: (i) the `.nc/.cs` sub-L1 advantage is deterministic (zero CI variance across 120 iterations on both architectures), so an architectural property not a scheduling artefact; (ii) the `.cs` L2-bound advantage (Section 4.4.1) is unpredicted by prior documentation; (iii) `.cg` never outperforms `.ca` for repeated access, and the realistic-kernel temporal-reuse result (Table 4.7) confirms the microbenchmark map transfers to production patterns.

5.3 Why Doesn't Triton Expose Them?

The dead zone has two distinct causes (Section 4.5), each implying a different intervention strategy. The *structural dead zone* (`matmul/attention`, `LDGSTS`) admits no PTX or SASS-level intervention on SM80+. The *choice-locked dead zone* (elementwise, `.nc` = optimal default) makes overrides counter-productive (26–135% cost). The conclusion applies to

Triton specifically, not to hand-written CUDA where modifiers are reachable via inline PTX; the decision guide in Figure 4.9 is directly actionable for custom-kernel developers.

5.4 Limitations

- **Aggregate validation.** Both instruments measure warp-level averages: agreement licenses ranking conditions by cache tier, not the tier of any individual load.
- **Single-block grid.** B-reload measurements use one block; multi-SM launches share L2 bandwidth across SMs and are not represented. Realistic CUDA kernels (Section 4.5.3) inherit the same constraint.
- **Triton version dependence.** Analysis is based on one Triton version; future compiler updates may shift defaults.
- **.cs L2-bound generality.** The Evict-First advantage was measured for sequential access; the random-gather kernel records -11.2% under .cs (Table 4.7). Irregular-access applications should bound expectations accordingly.

Chapter 6: Professional Considerations

6.1 Ethical Considerations

This project involves no human subjects, personal data, or sensitive information. All experiments use consumer GPU hardware and open-source software (CUDA toolkit, CuPy, Triton, NumPy). No adversarial or security-relevant use of computing resources is involved.

6.2 Legal Considerations

SASS disassembly was performed using `cuobjdump`, a tool distributed by NVIDIA within the CUDA toolkit for debugging and binary inspection. All disassembled kernels were compiled from the author’s own source code or from the open-source Triton project. No proprietary binaries or microcode are reproduced.

6.3 Reproducibility

Scripts version-controlled; results reported as medians with 95% bootstrap CIs (10,000 resamples) over thermal-throttling-resistant medians. SASS validation independently checks the PTX semantics.

Chapter 7: Conclusions and Recommendations

7.1 Conclusions

Five conclusions, one per claim from Chapter 1.

- 1. The four PTX cache modifiers are not characterised in the public literature; this study fills that gap.** Prior work [7, 6, 8, 9, 16] profiles the hierarchy without varying the modifier as an independent variable across sub-L1→L2→DRAM with hardware-counter cross-validation. Completed here on SM86 and SM75.
- 2. Two distinct timing regimes are required.** Per-instruction service time (pointer-chase, Section 4.2) and amortised throughput (B-reload, Section 4.3) are physically distinct regimes of the same memory pipeline; both must be measured to characterise modifier effects, and a single design covering both does not exist.
- 3. clock64 is a valid cache-tier profiler when cross-validated against NCU.** Tier agreement at every condition tested (Table 4.1); the $w = 6$ bank-stall is detected by timing while invisible to hit-rate, positive evidence that the joint design captures within-tier effects either instrument alone misses.
- 4. The performance map is working-set-conditional.** Three regimes with distinct modifier rankings (Tables 4.4 and 4.5); qualitative ranking preserved across SM86 and SM75. Cross-architecture magnitude scaling is per-modifier rather than uniform: `.cg` sub-L1 penalty scales with the L2/L1 latency ratio ($2.35\times$ vs. $\sim 1.82\times$); `.nc/.ca` sub-L1 advantage is flat at $\sim 26\%$; `.cs` L2-bound advantage runs the other way ($32\%/24\%$, mechanism-driven by the L2-traffic counter rather than latency).
- 5. Triton tuning is a dead zone; hand-written CUDA selection is actionable.** Triton matmul/attention is structurally locked (LDGSTS); elementwise is choice-locked. Hand-written CUDA reaches the magnitudes in Table 4.7 — large enough that modifier choice dominates measured throughput. The dead zone is a property of Triton, not the hardware.

7.2 Recommendations for Future Work

- 1. Multi-SM generalisation.** Repeat the modifier sweep with multi-block launches to characterise L2 bandwidth sharing across SMs and its effect on the collapse

threshold.

2. **Hopper and Ada Lovelace (SM90/SM89).** These architectures introduce LDGDEPBAR, the configurable L2 persistence API (`cudaAccessPolicyWindow`), and modified texture cache paths. The full dead-zone analysis should be repeated.
3. **Compiler .cs integration.** The .cs advantages (Tables 4.4 and 4.7) are not exploited by Triton; a heuristic emitting .cs when per-SM WS exceeds L1 could help streaming workloads (embedding-table lookups, data augmentation).
4. **Automatic modifier selection for hand-written CUDA.** The decision guide (Figure 4.9) and realistic-kernel results give sufficient conditions for static selection; an LLVM/nvcc pass analysing WS and access pattern at compile time would directly apply these findings.

Bibliography

- [1] cloudcores. CuASM: CUDA assembler and disassembler, 2021. Accessed April 2026.
- [2] Vasily Volkov. Better performance at lower occupancy. *Proceedings of the GPU Technology Conference (GTC)*, 2010. NVIDIA GPU Technology Conference.
- [3] Guoliang Wang, Huanqi Lin, Yidong Tian, Tianqi Chen, and Hao Zhang. CuAsmRL: Optimizing GPU SASS schedules via deep reinforcement learning, 2025.
- [4] Yueh-Hua Hsu, Changho Hong, Philippe Tillet, et al. KPerfIR: A compiler-centric performance IR for profiling GPU kernels, 2025.
- [5] NVIDIA Corporation. *PTX ISA Reference Manual, Release 8.5*, 2024. Released November 2024, Accessed April 2026.
- [6] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P. Scarpazza. Dissecting the NVIDIA Volta GPU architecture via microbenchmarking. In *arXiv preprint*, 2018.
- [7] Henry Wong, Misel-Myrto Papadopoulou, Madhi Sadooghi-Alvandi, and Andreas Moshovos. Demystifying GPU microarchitecture through microbenchmarking. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 235–246. IEEE, 2010.
- [8] Muhammad Muaaz Gul, Georg Hager, and Gerhard Wellein. Microbenchmarking modern NVIDIA GPU memory subsystems. *arXiv preprint*, 2020.
- [9] Da Yan, Wei Tan, Shivshankar Bhatt, Sidharath Sachidhanandan, and Dwarikanath Rao. Demystifying the NVIDIA ampere architecture through microbenchmarking and instruction-level analysis. In *arXiv preprint*, 2022.
- [10] Ervin Tasnádi. Nanobenchmarking: Cycle-accurate benchmarking of CUDA kernels. <https://etasnadi.com/2025/12/nanobenchmarking-cycle-accurate-benchmarking-of-cuda-kernels/>, December 2025. Accessed April 2026.
- [11] NVIDIA Corporation. *NVIDIA Nsight Compute Documentation*, 2024. Version 2024.x, Accessed April 2026.
- [12] Vasily Volkov. *Understanding Latency Hiding on GPUs*. PhD thesis, University of California, Berkeley, 2016. EECS-2016-143.

- [13] NVIDIA Corporation. *CUDA C++ Programming Guide*, 2024. Version 12.x, Accessed April 2026.
- [14] Philippe Tillet, H.T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10–19. ACM, 2019.
- [15] NVIDIA Corporation. *NVIDIA Ampere GA102 GPU Architecture Technical Brief*, 2021. Accessed April 2026.
- [16] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. NVIDIA A100 tensor core GPU: Performance and innovation. *IEEE Micro*, 41(2):29–35, 2021.
- [17] John D. C. Little. A proof for the queuing formula: $L = \lambda W$. *Operations Research*, 9(3):383–387, 1961.
- [18] Xinxin Mei and Xiaowen Chu. Dissecting GPU memory hierarchy through microbenchmarking. *IEEE Transactions on Parallel and Distributed Systems*, 28(6):1726–1741, 2017.

Chapter A: Experimental Parameter Reference

Table A.1: Complete parameter reference for all experiments.

Experiment	Parameter	Value
Cache-modifier sweep (SM86)	B_LEN	4096 floats (16 KB/warp)
	Warp counts	1,2,4,6,8,12,16,32
	Measurement reps	64
	Iterations	120
	Bootstrap samples	10 000
Cache-modifier sweep (SM75)	B_LEN	2048 floats (8 KB/warp)
	Warp counts	1,2,4,8,16,32
Warmup confound	Matrix size	$256 \times 256 \times 512$
	Fixed iterations	60
Bandwidth sweep	WS range	1 KB – 128 MB
	Iterations/point	20
	Modifiers	.ca, .cg
Latency sweep	Chain sizes	2 KB – 16 MB
	L2 flush	$2 \times 12 \text{CacheSize}$
NCU validation	NCU metric	<code>l1tex__t_sector_hit_rate.pct</code>
	clock64 metric	<code>median(.cg) / median(.ca)</code>
clock64 overhead	Null iterations	1 000
	Measured overhead	2 cy (p_{95} stable)

Chapter B: SASS Encoding Reference

Table B.1: Cache modifier bit encoding in LDG instruction qword2, bits [15:12].

bits [15:12]	SM86 opcode	SM75 opcode	PTX modifier
0xb	LDG.E.STRONG.SM	—	.ca
0x3	—	LDG.E.STRONG.CTA	.ca (SM75)
0xf	LDG.E.STRONG.GPU	LDG.E.STRONG.GPU	.cg
0x9	LDG.E.CONSTANT	—	.nc (SM86)
0x1	—	LDG.E.CONSTANT.SYS	.nc (SM75)
0x5	LDG.E.EF	LDG.E.EF	.cs

Triton gather kernel binary decode:

```
Instruction at offset 0x00e0:  
qword1 = 0x0000000402037981  
qword2 = 0x000ea2000c1e9900  
bits[15:12] = (0x000ea2000c1e9900 >> 12) & 0xf = 0x9  
Encoding 0x9 = LDG.E.CONSTANT -> .nc modifier
```

Chapter C: Raw Data Summary Tables

Table C.1: Four-modifier cache sweep results: SM86 RTX 3060 Ti. Median cycles per iteration with 95% bootstrap CI.

w	WS	.ca [lo,hi]	.cg [lo,hi]	.nc [lo,hi]	.cs [lo,hi]
1	16 KB	1701 [1701,1701]	3998 [3998,3998]	1259 [1259,1259]	1259 [1259,1259]
2	32 KB	1701 [1701,1705]	4009 [4008,4009]	1259 [1259,1259]	1259 [1259,1260]
4	64 KB	1710 [1709,1710]	4019 [4019,4020]	1260 [1260,1264]	1266 [1265,1272]
8	128 KB	2823 [2810,2840]	4772 [4766,4778]	2142 [2129,2152]	2533 [2518,2549]
16	256 KB	8154 [8147,8160]	8179 [8177,8180]	8148 [8142,8153]	5572 [5562,5579]
32	512 KB	16225[16219,16230]	16217[16213,16223]	16217[16209,16226]	14425[14399,14445]

Table C.2: Raw cross-instrument data for SM86: NCU L1 hit rate (both modifiers) and clock64 speed-up ratio. See Table 4.1 for annotated summary.

Warps	WS	NCU .ca hit (%)	NCU .cg hit (%)	clock64 Ratio	Note
1	16 KB	98.46	0.00	2.348	Deep sub-
2	32 KB	98.46	0.00	2.346	Sub-L1
4	64 KB	98.46	0.00	2.349	Sub-L1
6	96 KB	98.46	0.01	1.849	Bank stall: NCU hit unchar
8	128 KB	78.70	0.01	1.982	L1 bounda
12	192 KB	13.32	0.01	1.074	Mostly L
16	256 KB	1.13	0.01	1.002	L2-bound (coll
32	512 KB	0.02	0.01	1.000	Post-L2-saturation